

Project 2 Part A: Genetic Algorithm Design for Job Scheduling

Group 15

Aliyeh Haghollahi, Tapajit Chandra Paul.

1 Problem Representation and Definitions

The objective is to minimize the **makespan**, defined as the total time required to complete all jobs. A Genetic Algorithm (GA) is used, where each chromosome represents a valid job schedule (a permutation).

Chromosome definition:

A chromosome is a permutation of all job IDs (each job appears exactly once): $C = [J3, J1, J5, J2, J4]$

Definitions:

- J = number of jobs
- M = number of machines
- N = number of operations per job
- $\text{Time}[j][k]$ = processing time of job ID j at operation k (for $j \in \{1, \dots, J\}, k \in \{1, \dots, N\}$)

Machine assignment rule (round-robin): $\text{machine}(k) = ((k - 1) \bmod M) + 1$

Operation precedence constraint: Operation k of job j cannot start until operation $k - 1$ of the same job finishes.

Team contributions: Aliyeh Haghollahi: designed R2 makespan/fitness pseudocode and validated constraints. Tapajit Chandra Paul: designed R1 two-cut crossover pseudocode and permutation validity checks.

2 R1: Crossover Operation (Two-Cut Interval Crossover)

The crossover operator creates two children from two parent chromosomes using two cut points and a hole mechanism to avoid duplicate jobs, while preserving relative order from the other parent.

Input assumptions: P1 and P2 are valid chromosomes (permutations) containing each job ID exactly once, with job IDs in the range $1..J$. The procedure must produce two valid children that also contain each job exactly once.

Pseudo Code (Permutation-Safe, Two Children)

```
PROCEDURE Crossover(P1, P2)
  J ← length(P1)

  # Step 1: choose two distinct cut points
  cut1 ← RANDOM(1, J)
  REPEAT cut2 ← RANDOM(1, J) UNTIL cut2 ≠ cut1
  IF cut1 > cut2 THEN SWAP(cut1, cut2) END IF

  # Helper: BuildChild(KeepParent, FillParent)
```

```
PROCEDURE BuildChild(KP, FP)
  Child ← EMPTY array size J
  # Copy middle segment from KP
  FOR i = cut1 TO cut2 DO
    Child[i] ← KP[i]
  END FOR

  # Mark holes in FP for items already used
  in Child[cut1..cut2]
  FPholes ← copy of FP
  FOR i = 1 TO J DO
    IF FPholes[i] exists in KP[cut1..cut2] THEN
      FPholes[i] ← H
    END IF
  END FOR

  # Fill remaining slots by scanning FP
  left-to-right (ignoring holes)
  pos ← 1
  FOR i = 1 TO J DO
    IF FPholes[i] ≠ H THEN
      # find next empty position outside the cut
      segment
      WHILE Child[pos] ≠ EMPTY DO
        pos ← pos + 1
      END WHILE
      IF pos > J THEN BREAK END IF
      Child[pos] ← FPholes[i]
    END IF
  END FOR

  RETURN Child
END PROCEDURE

# Step 2: build both children
Child1 ← BuildChild(P1, P2)
Child2 ← BuildChild(P2, P1)
```

```
RETURN Child1, Child2
END PROCEDURE
```

3 R2: Makespan and Fitness Computation

The makespan is computed by simulating the schedule: each job's operations are executed in order, and each operation must wait for both (1) its assigned machine to be free and (2) the previous operation of that job to finish.

Input assumptions: C is a valid chromosome (permutation of job IDs $1..J$). The matrix $\text{Time}[j][k]$ is defined for every job j and operation k , and all processing times are known in advance.

Pseudo Code (Generic for Any J, M, N)

```
PROCEDURE ComputeMakespan(C, Time, M, N)

  J ← length(C)
```

```

# machine availability times
FOR m = 1 TO M DO
  machineFree[m] ← 0
END FOR

# job readiness times (indexed by job ID:
1..J)
FOR jobId = 1 TO J DO
  jobReady[jobId] ← 0
END FOR

makespan ← 0

# schedule jobs in chromosome order
FOR idx = 1 TO J DO
  jobId ← C[idx]

  FOR k = 1 TO N DO
    # round-robin machine assignment for
operation k
    m ← ((k - 1) mod M) + 1

    # operation waits for machine and previous
op of same job
    start ← MAX(machineFree[m],
jobReady[jobId])
    finish ← start + Time[jobId][k]

    machineFree[m] ← finish
    jobReady[jobId] ← finish

    IF finish > makespan THEN makespan ←
finish END IF
  END FOR
END FOR

RETURN makespan
END PROCEDURE

```

- Makespan correctly measures the overall completion time across all machines and jobs.
- The pseudocode is fully generic and works for any number of jobs, machines, and operations.

Fitness Function

```

PROCEDURE ComputeFitness(C, Time, M, N)
  ms ← ComputeMakespan(C, Time, M, N)
  # smaller makespan => larger fitness
  fitness ← 1 / ms
  RETURN fitness
END PROCEDURE

```

4 Notes

This design satisfies the project constraints:

- Machines never execute more than one operation at a time (enforced by machineFree).
- Job operation precedence is preserved (enforced by jobReady).
- Crossover produces valid permutations (no duplicates due to the hole mechanism).